Programming Languages and Environments (Lecture 19)

LEI - Licenciatura em Engenharia Informática

João Costa Seco (joao.seco@fct.unl.pt)

NOVA SCHOOL OF SCIENCE & TECHNOLOGY

Syllabus

- Language support for concurrency
- I/O asynchronous, Promises and Futures

Concurrency

- Ability to perform multiple computations that overlap in time, without requiring them to run sequentially.
  - Graphical interfaces: to ensure quick responses to user actions
  - Spreadsheets: to recalculate all formulas without interruptions.
  - Web browsers: to load and display pages incrementally.
  - Servers: to handle multiple clients without making them wait.
  - GPU: massive parallelism
- How?
  - Interleaving: switching rapidly between active computations
  - Parallelism: using multiple physical processors (multi-core).
- Preemptive / Collaborative

Non-determinism

- A program that can have different results every time it is executed is a non-deterministic program.
- Introducing concurrency also introduces non-determinism, by not fixing an order by which the operations are executed.
- When two imperative programs share state variables, the possible changes to that state become indeterminate. Thus, it is not easy to predict the exact behaviour of these programs.
- The interactions/interference between programs can be benign or malign (race conditions).
- Functional languages make it easier to reason about programs, seeing as a pure expression always denotes the same value.

Promises and Futures

- They represent computations that have not yet finished but will eventually denote a value at some point in the future (deferred)
- Typically associated with a computation concurrent with the main thread.

```
async function getNames() {
  const response = await fetch('https://server.com/users')
  const data = await response.json()
  return data.map(user ⇒ user.name)
}
```

- Async (Jane Street) and Lwt (Ocsigen) are two popular libraries that implement asynchronous computation in OCaml.

Promises (Lwt)

- They are data abstractions for a model of asynchronous computation.

- Promises are references, their value can change.
- When created, they contain nothing.
- A promise can be fulfilled and assigned a value.
- A promise can be rejected (assigned an exception)
- It both cases, the promise is said to be resolved.

Signature of the Promise module

```
(** A signature for Lwt-style promises, with better names *)
module type PROMISE = sig
  type 'a state =
    | Pending
    | Fulfilled of 'a
    | Rejected of exn

  type 'a promise

  type 'a resolver

  (** [make ()] is a new promise and resolver. The promise is pending. *)
  val make : unit → 'a promise * 'a resolver

  (** [return x] is a new promise that is already fulfilled with value
      [ x ]. *)
  val return : 'a → 'a promise

  (** [state p] is the state of the promise *)
  val state : 'a promise → 'a state

  (** [fulfill r x] fulfills the promise [p] associated with [r] with
      value [ x ], meaning that [state p] will become [Fulfilled x].
      Requires: [p] is pending. *)
  val fulfill : 'a resolver → 'a → unit

  (** [reject r x] rejects the promise [p] associated with [r] with
      exception [ x ], meaning that [state p] will become [Rejected x].
      Requires: [p] is pending. *)
  val reject : 'a resolver → exn → unit
end
```

[1] ✓ 0.0s OCaml

Implementation of the Promise module

```
module Promise : PROMISE = struct
  type 'a state =
    | Pending
    | Fulfilled of 'a
    | Rejected of exn

  type 'a promise = 'a state ref

  type 'a resolver = 'a promise

  (** [write_once p s] changes the state of [p] to be [s]. If [p] and
      [s] are both pending, that has no effect. Raises: [Invalid_arg] if
      the state of [p] is not pending. *)
  let write_once p s =
    if !p = Pending then p := s else invalid_arg "cannot write twice"

  let make () =
    let p = ref Pending in
    (p, p)

  let return x = ref (Fulfilled x)

  let state p = !p

  let fulfill r x = write_once r (Fulfilled x)

  let reject r x = write_once r (Rejected x)
end
```

[4] ✓ 0.0s OCaml

```
module Promise : PROMISE
```

I/O Asynchronous

```
let file = "example.dat"
let message = "Hello!"

let () =
  let oc = open_out file in
  Printf.fprintf oc "%s\n" message;
  close_out oc;

  let ic = open_in file in
  try
    let line = input_line ic in
    print_endline line;
    flush stdout;
    close_in ic
  with e →
    close_in_noerr ic;
    raise e
```

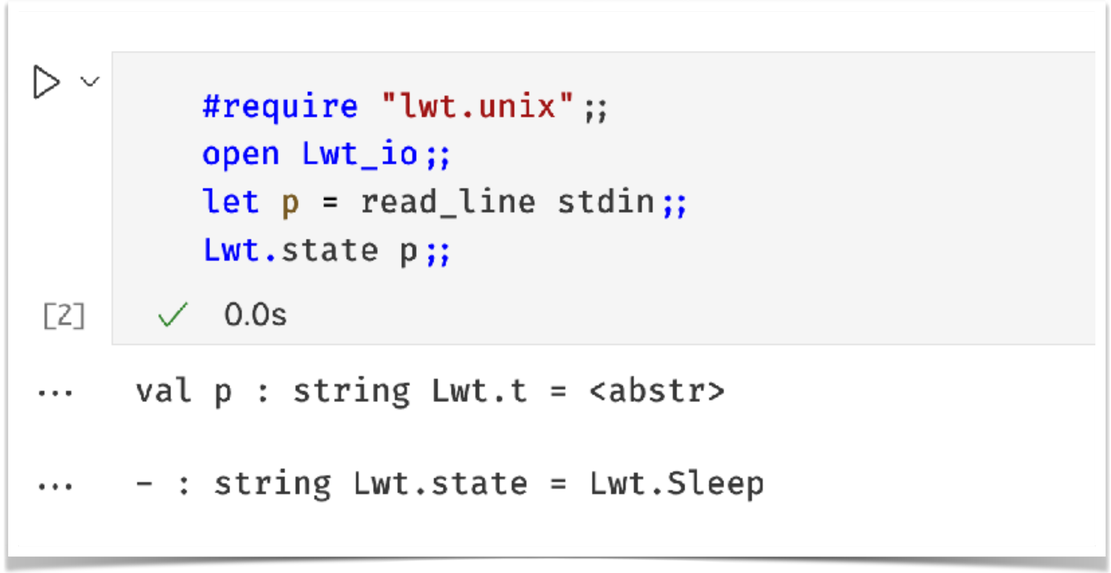
```
# ignore(input_line stdin); print_endline "done";;
<type your own input here>
done
- : unit = ()
```

https://ocaml.org/docs/file-manipulation

https://cs3110.github.io/textbook/chapters/ds/promises.html

I/O Asynchronous

- The I/O primitives do not block while waiting for it to terminate.
- Their value denotes a promise.
- The type of the promise is masked by the `utop`, but it is in fact `string Lwt.t`



A notação desta imagem é a notação nativa do módulo lwt

Callbacks

- A callback function of a promise is a function that is called when the promise is resolved.
- A callback function is attached to a promise using the **`bind`** function.

```
[3]  let print_the_string str = Lwt_io.printf "The string is: %S\n" str;;

     Lwt.bind p print_the_string

...  val print_the_string : string → unit Lwt.t = <fun>
```

```
[3]  Lwt.bind
     ✓ 0.0s

...  - : 'a Lwt.t → ('a → 'b Lwt.t) → 'b Lwt.t = <fun>
```

Callbacks 2

- Multiple asynchronous operations can be chained to maintain determinism/ sequentiality.
- The **`bind`** function combines the **`run`** function, waits for the total result, and returns a final value.

```
[4]
open Lwt_io

let p =
  Lwt.bind (read_line stdin) (fun s1 →
    Lwt.bind (read_line stdin) (fun s2 →
      Lwt_io.printf "%s\n" (s1^s2)))

let _ = Lwt_main.run p
```

```
[5]  ✓ 0.0s
Lwt.bind;;

Lwt_main.run
- : 'a Lwt.t → ('a → 'b Lwt.t) → 'b Lwt.t = <fun>
- : 'a Lwt.t → 'a = <fun>
```

```
utop # #use "teoricas/LAP2024-15/promises.ml";;
val p : unit Lwt.t = <abstr>
one
two
- : unit = ()
onetwo
```

Callbacks 2

- Multiple asynchronous operations can be chained to maintain determinism/ sequentiality.
- The **`bind`** function combines the **`run`** function, waits for the total result, and returns a final value.

>>=

```
open Lwt_io
open Lwt.Infix

let p =
  read_line stdin >>= fun s1 ->
  read_line stdin >>= fun s2 ->
  Lwt_io.printf "%s\n" (s1^s2)

let _ = Lwt_main.run p
```

```
Lwt.bind;;

Lwt_main.run
```

[5] ✓ 0.0s

```
- : 'a Lwt.t → ('a → 'b Lwt.t) → 'b Lwt.t = <fun>

- : 'a Lwt.t → 'a = <fun>
```

```
utop # #use "teoricas/LAP2024-15/promises.ml";;
val p : unit Lwt.t = <abstr>
one
two
- : unit = ()
onetwo
```

Callbacks 2

- Multiple asynchronous operations can be chained to maintain determinism/ sequentiality.
- The **`bind`** function combines the r… function, waits for the total result, a… returns a final value.

```
>>=
```

```
open Lwt_io
open Lwt.Infix

let p =
  read_line stdin >>= fun s1 →
  read_line stdin >>= fun s2 →
```

```
- : unit = ()
onetwo
```

```
function getNames() {
  return fetch('https://server.com/users')
              .then( response ⇒ response.json() )
              .then( data ⇒ data.map(user ⇒ user.name));
}

getNames().then( names ⇒ console.log(names) );
```

```
Lwt.bind;;

Lwt_main.run
```

```
[5]  ✓ 0.0s
```

```
- : 'a Lwt.t → ('a → 'b Lwt.t) → 'b Lwt.t = <fun>

- : 'a Lwt.t → 'a = <fun>
```

Callbacks implemented

```
(** A signature for Lwt-style promises, with better names *)
module type PROMISE = sig
  type 'a state =
    | Pending
    | Fulfilled of 'a
    | Rejected of exn

  type 'a promise

  type 'a resolver

  (** [make ()] is a new promise and resolver. The promise is pending. *)
  unit -> 'a promise * 'a resolver
  val make : unit → 'a promise * 'a resolver

  (** [return x] is a new promise that is already fulfilled with value [ x ]. *)
  'a -> 'a promise
  val return : 'a → 'a promise

  (** [state p] is the state of the promise *)
  'a promise -> 'a state
  val state : 'a promise → 'a state

  (** [fulfill r x] resolves the promise [p] associated with [r] with
      value [ x ], meaning that [state p] will become [Fulfilled x].
      Requires: [p] is pending. *)
  'a resolver -> 'a -> unit
  val fulfill : 'a resolver → 'a → unit

  (** [reject r x] rejects the promise [p] associated with [r] with
      exception [ x ], meaning that [state p] will become [Rejected x].
      Requires: [p] is pending. *)
  'a resolver -> exn -> unit
  val reject : 'a resolver → exn → unit

  (** [p >>= c] registers callback [c] with promise [p].
      When the promise is fulfilled, the callback will be run
      on the promises's contents.  If the promise is never
      fulfilled, the callback will never run. *)
  'a promise -> ('a -> 'b promise) -> 'b promise
  val ( >>= ) : 'a promise → ('a → 'b promise) → 'b promise
end
```

https://cs3110.github.io/textbook/chapters/ds/promises.html

Callbacks implemented

```
module Promise : PROMISE = struct
  type 'a state = Pending | Fulfilled of 'a | Rejected of exn

  (** RI (representation invariant): the input may not be [Pending] *)
  type 'a handler = 'a state → unit

  (** RI: if [state <> Pending] then [handlers = []]. *)
  type 'a promise = {
    mutable state : 'a state;
    mutable handlers : 'a handler list
  }
```

Callbacks implemented

```
('a state -> unit) -> 'a promise -> unit
let enqueue
      (handler : 'a state → unit)
      (promise : 'a promise) : unit
    =
    promise.handlers ← handler :: promise.handlers

type 'a resolver = 'a promise

(** [write_once p s] changes the state of [p] to be [s].  If [p] and [s]
      are both pending, that has no effect.
      Raises: [Invalid_arg] if the state of [p] is not pending. *)
'a resolver -> 'a state -> unit
let write_once p s =
  if p.state = Pending
  then p.state ← s
  else invalid_arg "cannot write twice"
```

Callbacks implemented

```
unit -> 'a resolver * 'a resolver
let make () =
  let p = {state = Pending; handlers = []} in
  p, p

'a -> 'a resolver
let return x =
  {state = Fulfilled x; handlers = []}

'a resolver -> 'a state
let state p = p.state

(** requires: [st] may not be [Pending] *)
'a resolver -> 'a state -> unit
let resolve (r : 'a resolver) (st : 'a state) =
  assert (st <> Pending);
  let handlers = r.handlers in
  r.handlers ← [];
  write_once r st;
  List.iter (fun f → f st) handlers
```

Callbacks implemented

```
'a resolver -> exn -> unit
let reject r x =
  resolve r (Rejected x)

'a resolver -> 'a -> unit
let fulfill r x =
  resolve r (Fulfilled x)

'a resolver -> 'a handler
let handler (resolver : 'a resolver) : 'a handler
  = function
    | Pending → failwith "handler RI violated"
    | Rejected exc → reject resolver exc
    | Fulfilled x → fulfill resolver x
```

Callbacks implemented

```
('a -> 'b resolver) -> 'b resolver -> 'a handler
let handler_of_callback
    (callback : 'a → 'b promise)
    (resolver : 'b resolver) : 'a handler
  = function
    | Pending → failwith "handler RI violated"
    | Rejected exc → reject resolver exc
    | Fulfilled x →
      let promise = callback x in
      match promise.state with
      | Fulfilled y → fulfill resolver y
      | Rejected exc → reject resolver exc
      | Pending → enqueue (handler resolver) promise

'a resolver -> ('a -> 'b resolver) -> 'b resolver
let ( >>= )
    (input_promise : 'a promise)
    (callback : 'a → 'b promise) : 'b promise
  =
  match input_promise.state with
  | Fulfilled x → callback x
  | Rejected exc → {state = Rejected exc; handlers = []}
  | Pending →
    let output_promise, output_resolver = make () in
    enqueue (handler_of_callback callback output_resolver) input_promise;
    output_promise
end
```

Next lecture

- Language support for concurrency (part 2)
  - OCaml 5.0
  - Go
  - Rust

Language and Compiler Design (MEI)

- A deep dive into language design mechanisms
  - Source language with mutability, closures, and pattern matching
  - Static and dynamic semantics of languages, new abstractions, language-based analyses.
- Modern interpreter and compiler construction and language optimisation techniques.
- Possible Target architectures
  - LLVM + clang backend,
  - RISC5 - embedded ESP32-C,
  - WASM - browser + RUST or Kotlin
- Compiler implementation language options: OCaml, Scala, or Rust